

Assignment 2: Coding Basics

Shuer Li

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1. Generate a sequence of numbers from one to 55, increasing by fives  
seq(1,55,5)
```

```
## [1] 1 6 11 16 21 26 31 36 41 46 51
```

```
sl_sequence <- seq(1,55,5)  
sl_sequence
```

```
## [1] 1 6 11 16 21 26 31 36 41 46 51
```

```
#2. Compute the mean and median of this sequence  
mean(sl_sequence)
```

```
## [1] 26
```

```
median(sl_sequence)
```

```
## [1] 26
```

```
#3. Ask R to determine whether the mean is greater than the median
```

```
mean(sl_sequence) > median(sl_sequence)
```

```
## [1] FALSE
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
#5. Create three vectors
```

```
students_names <- c("Amy","Shirly","Jack","Tom") # Vector type: Character  
students_names
```

```
## [1] "Amy"      "Shirly"    "Jack"      "Tom"
```

```
test_scores <- c(59,92,62,89) # Vector type: Numeric  
test_scores
```

```
## [1] 59 92 62 89
```

```
on_scholarship <- c(FALSE,TRUE,FALSE,TRUE) # Vector type: Logical  
on_scholarship
```

```
## [1] FALSE TRUE FALSE TRUE
```

```
#7. Combine each of the vectors into a data frame
```

```
students_data <- data.frame(students_names,test_scores,on_scholarship)  
students_data
```

```
##   students_names test_scores on_scholarship  
## 1         Amy         59         FALSE  
## 2        Shirly         92          TRUE  
## 3         Jack         62         FALSE  
## 4          Tom         89          TRUE
```

```
#8. Label the columns of the data frame with informative titles
```

```
names(students_data) <- c("Name","Scores","Scholarship")  
students_data
```

```
##      Name Scores Scholarship
## 1     Amy     59         FALSE
## 2  Shirly     92          TRUE
## 3    Jack     62         FALSE
## 4     Tom     89          TRUE
```

9. QUESTION: How is this data frame different from a matrix?

Answer: A data frame can store different data types in its columns, such as numeric, character, logical, while a matrix can only contain one type of data.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word "Pass"; otherwise print the word "Fail".
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using `if`...`else`
pass_fail_if <- function(score) {
  if (score > 50) {
    print("Pass")
  }
  else {
    print("Fail")
  }
}

#11. Create a function using `ifelse()`
pass_fail_ifelse <- function(score) {
  ifelse(score > 50, "Pass", "Fail")
}

#12a. Run the first function with the value 52.5
pass_fail_if(52.5)
```

```
## [1] "Pass"
```

```
#12b. Run the second function with the value 52.5
pass_fail_ifelse(52.5)
```

```
## [1] "Pass"
```

```
#13a. Run the first function with the vector of test scores
#pass_fail_if(test_scores) #This will cause an error because if...else only handles a single value

#13b. Run the second function with the vector of test scores
pass_fail_ifelse(test_scores)
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: The `ifelse` worked because it is vectorized. It can evaluate each element of a vector, while `if...else` only evaluates one condition at a time, so it cannot handle vectors.

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)